

Digital Forensics Lab Record

Python Programs and Output — Experiments 1–28

Experiment 1: Write a Python program to demonstrate Locard's Exchange

Principle using file timestamps(metadata trace demo).

```
import os
import shutil
import time
# Create an "original" piece of evidence
with open("evidence_original.txt", "w") as f:
    f.write("Case File #001 - Suspect device log")
orig_stat = os.stat("evidence_original.txt")
print("Original file - created (ctime):", time.ctime(orig_stat.st_ctime))
print("Original file - modified (mtime):", time.ctime(orig_stat.st_mtime))
# Simulate an investigator copying the file
time.sleep(1)
shutil.copy2("evidence_original.txt", "evidence_copy.txt")
copy_stat = os.stat("evidence_copy.txt")
print("\nCopied file - created (ctime):", time.ctime(copy_stat.st_ctime))
print("Copied file - modified (mtime):", time.ctime(copy_stat.st_mtime))
print("\nLocard's Exchange Principle:")
print("The copy operation itself created a new")
print("ctime on the destination file - every interaction leaves a trace.")
```

```
Original file - created (ctime): Thu Jul 16 05:13:46 2026
Original file - modified (mtime): Thu Jul 16 05:13:46 2026
```

```
Copied file - created (ctime): Thu Jul 16 05:13:47 2026
Copied file - modified (mtime): Thu Jul 16 05:13:46 2026
```

```
Locard's Exchange Principle:
The copy operation itself created a new
ctime on the destination file - every interaction leaves a trace.
```

Experiment 2: Write a Python program to verify the chain of custody of digital evidence using file hashing.

```
import hashlib
import datetime

# Function to calculate SHA-256 hash
def sha256_of(filename):
    with open(filename, "rb") as f:
        return hashlib.sha256(f.read()).hexdigest()

# Step 1: Create evidence
with open("evidence.txt", "w") as f:
    f.write("Suspect chat log: meeting at 10pm, bring the drive.")

# Calculate initial hash
seizure_hash = sha256_of("evidence.txt")

# Chain of custody log
custody_log = []

custody_log.append(
    f"{datetime.datetime.now()} - SEIZED by Officer A - hash={seizure_hash}"
)

# Step 2: Verify evidence integrity
handoff_hash = sha256_of("evidence.txt")

if handoff_hash == seizure_hash:
    custody_log.append(
        f"{datetime.datetime.now()} - RECEIVED by Analyst B - integrity VERIFIED"
    )
else:
    custody_log.append(
        f"{datetime.datetime.now()} - RECEIVED by Analyst B - integrity FAILED (tampered!)"
    )

print("=== Chain of Custody Log ===")
for entry in custody_log:
    print(entry)

=== Chain of Custody Log ===
2026-07-16 05:27:50.643435 - SEIZED by Officer A - hash=b7eb05cf41efbcc50adc36b12679ce4f12de58460e59500eb6bb62858810f84c
2026-07-16 05:27:50.644258 - RECEIVED by Analyst B - integrity VERIFIED
```

Experiment 3: Write a Python program to detect phishing emails based on suspicious indicators in the sender, subject, and body.

```
import re

def check_email(sender, subject, body):
    flags = []

    if re.search(r"(urgent|verify your account|suspended|click here)", body, re.IGNORECASE):
        flags.append("Urgency/pressure language detected")

    if re.search(r"\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}", body):
        flags.append("Raw IP address link found in body")

    domain = sender.split("@")[-1]

    if any(k in domain.lower() for k in ["secure", "verify", "update"]) and "@gmail" not in sender:
        flags.append("Suspicious sender domain naming pattern")

    return flags

emails = [
    (
        "support@paypal.com",
        "Your monthly statement",
        "Please find your statement attached."
    ),
    (
        "alert@paypal-secure-verify.com",
        "URGENT: Verify your account",
        "Click here: http://192.168.10.5/login"
    )
]

for sender, subject, body in emails:
    issues = check_email(sender, subject, body)
    verdict = "PHISHING SUSPECTED" if issues else "Looks legitimate"

    print("\nFrom:", sender)
    print("Subject:", subject)
    print("Verdict:", verdict)

    for issue in issues:
        print("-", issue)

From: support@paypal.com
Subject: Your monthly statement
Verdict: Looks legitimate

From: alert@paypal-secure-verify.com
Subject: URGENT: Verify your account
Verdict: PHISHING SUSPECTED
- Urgency/pressure language detected
- Raw IP address link found in body
- Suspicious sender domain naming pattern
```

Experiment 4: Write a Python program to perform a DNS lookup for a domain as a basic OSINT reconnaissance technique.

```
import socket
domains = [
    "www.google.com",
    "www.python.org",
    "notarealdomain12345.com"
]
print("OSINT Domain Reconnaissance")
for d in domains:
    try:
        ip = socket.gethostbyname(d)
        print(f"{d:30s} -> {ip}")
    except socket.gaierror:
        print(f"{d:30s} -> Could not resolve (invalid/unreachable)")

OSINT Domain Reconnaissance
www.google.com           -> 142.251.151.119
www.python.org           -> 151.101.0.223
notarealdomain12345.com  -> Could not resolve (invalid/unreachable)
```

Experiment 5: Write a Python program to generate a structured report covering the five-stage digital forensic investigation lifecycle.

```
import datetime

def investigation_report(case_id, evidence_list):
    stages = {
        "Identification": f"Incident reported for case {case_id}. Devices/logs identified for review.",
        "Collection": f"{len(evidence_list)} item(s) collected: {' '.join(evidence_list)}",
        "Preservation": "All items hashed (SHA-256) and stored in a write-protected evidence folder.",
        "Analysis": "Log files and file metadata examined for indicators of compromise.",
        "Reporting": "Findings compiled into a structured forensic report for legal review."
    }

    print(f"=== Investigation Lifecycle: Case {case_id} ===")
    print(f"Generated: {datetime.datetime.now()}\n")

    for stage, detail in stages.items():
        print(f"[{stage}]")
        print(f"    {detail}\n")

investigation_report(
    "CASE-2026-014",
    [
        "laptop_disk_image.dd",
        "router_traffic.pcap",
        "email_headers.txt"
    ]
)

=== Investigation Lifecycle: Case CASE-2026-014 ===
Generated: 2026-07-16 05:43:16.508220

[Identification]
    Incident reported for case CASE-2026-014. Devices/logs identified for review.

[Collection]
    3 item(s) collected: laptop_disk_image.dd, router_traffic.pcap, email_headers.txt

[Preservation]
    All items hashed (SHA-256) and stored in a write-protected evidence folder.

[Analysis]
    Log files and file metadata examined for indicators of compromise.

[Reporting]
    Findings compiled into a structured forensic report for legal review.
```

Experiment 6: Write a Python program to create a bit-stream (forensic) copy of a file and verify its integrity using hashing.

```
import hashlib

# Function to calculate SHA-256 hash
def sha256_of(filename):
    with open(filename, "rb") as f:
        return hashlib.sha256(f.read()).hexdigest()

# Create a sample "disk" file
# This simulates a small storage device
with open("original_disk.img", "wb") as f:
    f.write(b"HEADER" + bytes(range(256)) * 4 + b"FOOTER")

# Step 1: Create a bit-for-bit forensic copy
with open("original_disk.img", "rb") as src:
    with open("forensic_copy.img", "wb") as dst:
        dst.write(src.read())

# Step 2: Verify the copy using SHA-256 hashing
original_hash = sha256_of("original_disk.img")
copy_hash = sha256_of("forensic_copy.img")

# Display the results
print("Original image hash:", original_hash)
print("Forensic copy hash :", copy_hash)

if original_hash == copy_hash:
    print("Match: VERIFIED - exact bit-stream copy")
else:
    print("MISMATCH - copy corrupted")

Original image hash: d50630bee2bf9926344781042d10cb09377730818efda8e80f9eeec21008a9b0
Forensic copy hash : d50630bee2bf9926344781042d10cb09377730818efda8e80f9eeec21008a9b0
Match: VERIFIED - exact bit-stream copy
```

Experiment 7: Write a Python program to identify a file's true type from its byte-level signature (magic number).

```
SIGNATURES = {
    b"\xFF\xD8\xFF": "JPEG image",
    b"\x89PNG": "PNG image",
    b"%PDF": "PDF document",
    b"PK\x03\x04": "ZIP archive (or .docx/.xlsx)",
}

# Function to identify file type
def identify_file(path):
    with open(path, "rb") as f:
        header = f.read(8)

    for sig, filetype in SIGNATURES.items():
        if header.startswith(sig):
            return filetype

    return "Unknown file type"

# Create sample files with fake/renamed extensions
# Testing signature detection

# Simulate a JPEG file renamed as .txt
with open("photo.txt", "wb") as f:
    f.write(b"\xFF\xD8\xFF\xE0" + b"\x00" * 20)

# Simulate a PDF file renamed as .dat
with open("document.dat", "wb") as f:
    f.write(b"%PDF-1.4" + b"\x00" * 20)

# Identify the actual file types
for filename in ["photo.txt", "document.dat"]:
    print(f"{filename:15s} -> Actual type: {identify_file(filename)}")

photo.txt      -> Actual type: JPEG image
document.dat   -> Actual type: PDF document
```

Experiment 8: Write a Python program to simulate recovery of deleted files by scanning storage for recognizable byte patterns.

```
import os

# Step 1: Create a file, then "delete" it
# This simulates accidental or malicious deletion
with open("secret_note.txt", "w") as f:
    f.write("MEETING_POINT: warehouse 7, 11pm")

# Read and store the original file contents
with open("secret_note.txt", "rb") as f:
    original_bytes = f.read()

# Delete the file
os.remove("secret_note.txt")

print("File 'secret_note.txt' deleted.")
print("Exists on disk now?", os.path.exists("secret_note.txt"))

# Step 2: Simulate raw disk scanning
# In real forensics, tools can scan unallocated space
# for recognizable byte patterns.

disk_buffer = original_bytes + b"\x00" * 50

# Search for a known marker
marker = b"MEETING_POINT"
index = disk_buffer.find(marker)

# Step 3: Recover the data
if index != -1:
    recovered = disk_buffer[index:].split(b"\x00")[0]

    with open("recovered_note.txt", "wb") as f:
        f.write(recovered)

    print("Recovered content:", recovered.decode())
else:
    print("No recoverable data found.")

File 'secret_note.txt' deleted.
Exists on disk now? False
Recovered content: MEETING_POINT: warehouse 7, 11pm
```


Experiment 9: Write a Python program to analyze a system login log and detect brute-force intrusion attempts.

```
import re
from collections import Counter

# Create the sample system log
with open("login_attempts.log", "w") as f:
    f.write("2026-07-08 09:00:01 LOGIN SUCCESS user=alice ip=10.0.0.5\n")
    f.write("2026-07-08 09:01:15 LOGIN FAILED user=admin ip=203.0.113.99\n")
    f.write("2026-07-08 09:01:20 LOGIN FAILED user=admin ip=203.0.113.99\n")
    f.write("2026-07-08 09:01:25 LOGIN FAILED user=admin ip=203.0.113.99\n")
    f.write("2026-07-08 09:01:30 LOGIN FAILED user=admin ip=203.0.113.99\n")
    f.write("2026-07-08 09:01:35 LOGIN FAILED user=admin ip=203.0.113.99\n")
    f.write("2026-07-08 09:02:00 LOGIN SUCCESS user=bob ip=10.0.0.8\n")

# Count failed login attempts by IP address
failed_by_ip = Counter()

with open("login_attempts.log") as f:
    for line in f:
        if "LOGIN FAILED" in line:
            match = re.search(r"ip=(\S+)", line)

            if match:
                failed_by_ip[match.group(1)] += 1

# Display forensic analysis results
print("System log forensic analysis - failed logins by source IP:")

for ip, count in failed_by_ip.items():
    print(f" {ip}: {count} failed attempts")

    if count >= 5:
        print(f" -> Evidence of brute-force intrusion attempt from {ip}")

System log forensic analysis - failed logins by source IP:
203.0.113.99: 5 failed attempts
-> Evidence of brute-force intrusion attempt from 203.0.113.99
```

Experiment 10: Write a Python program to parse raw email headers and trace indicators of a spoofed sender.

```
raw_header = """Delivered-To: victim@example.com
Received: from mail-relay-99.suspicious-host.ru (unknown [45.33.32.156])
by mx.example.com; Tue, 08 Jul 2026 09:15:00 +0000
From: "Bank Support"
Reply-To: attacker@totallynotscam.net
Subject: Urgent: Verify your account
"""

def analyze_header(header_text):
    findings = []

    for line in header_text.splitlines():
        # Check for suspicious relay server
        if line.startswith("Received:") and "suspicious" in line.lower():
            findings.append(
                f"Suspicious relay server found: {line.strip()}"
            )

        # Check for Reply-To header
        if line.startswith("Reply-To:"):
            findings.append(
                f"Reply-To differs from sender - possible spoofing: {line.strip()}"
            )

        # Check for suspicious sender domain
        if line.startswith("From:") and "-secure-verify" in line:
            findings.append(
                f"Sender domain uses suspicious naming: {line.strip()}"
            )

    return findings

print("Email Header Forensic Analysis")

for finding in analyze_header(raw_header):
    print(" -", finding)

Email Header Forensic Analysis
- Suspicious relay server found: Received: from mail-relay-99.suspicious-host.ru (unknown [45.33.32.156])
- Sender domain uses suspicious naming: From: "Bank Support" <support@paypal-secure-verify.com>
- Reply-To differs from sender - possible spoofing: Reply-To: attacker@totallynotscam.net
```

Experiment 11: Write a Python program to analyze a simulated network traffic log and detect port-scanning behaviour.

```
with open("traffic.log", "w") as f:
    f.write("10.0.0.9 -> 10.0.0.5:22 SYN\n")
    f.write("10.0.0.9 -> 10.0.0.5:23 SYN\n")
    f.write("10.0.0.9 -> 10.0.0.5:25 SYN\n")
    f.write("10.0.0.9 -> 10.0.0.5:80 SYN\n")
    f.write("10.0.0.9 -> 10.0.0.5:443 SYN\n")
    f.write("10.0.0.20 -> 10.0.0.5:80 SYN\n")
    f.write("10.0.0.20 -> 10.0.0.5:80 ACK\n")

# Count distinct destination ports probed by each source IP
ports_by_source = {}

with open("traffic.log") as f:
    for line in f:
        parts = line.split()

        src = parts[0]
        dst_port = parts[2].split(":")[1]

        ports_by_source.setdefault(src, set()).add(dst_port)

# Display forensic analysis
print("Network traffic forensic analysis:")

for src, ports in ports_by_source.items():
    print(
        f" {src}: contacted {len(ports)} distinct port(s) -> {sorted(ports)}"
    )

    if len(ports) >= 4:
        print(
            f" -> ALERT: {src} shows port-scanning behaviour"
        )

Network traffic forensic analysis:
10.0.0.9: contacted 5 distinct port(s) -> ['22', '23', '25', '443', '80']
-> ALERT: 10.0.0.9 shows port-scanning behaviour
10.0.0.20: contacted 1 distinct port(s) -> ['80']
```

Experiment 12: Write a Python program to extract and display EXIF metadata embedded in an image file.

```
from PIL import Image
from PIL.ExifTags import TAGS

# Install Pillow once if needed:
# pip install Pillow

# Function to create a sample image
def create_sample_image(path):
    # Creates a small test image
    # A real photo may contain EXIF metadata
    img = Image.new("RGB", (100, 100), color="blue")
    img.save(path)

# Function to display image metadata
def show_metadata(path):
    img = Image.open(path)

    print("File:", path)
    print("Format:", img.format)
    print("Size:", img.size)
    print("Mode:", img.mode)

    # Extract EXIF metadata
    exif_data = img._getexif() if hasattr(img, "_getexif") else None

    if exif_data:
        for tag_id, value in exif_data.items():
            tag = TAGS.get(tag_id, tag_id)
            print(f" {tag}: {value}")
    else:
        print(" No EXIF metadata found (this sample image has none).")

    print("Real photos from phones/cameras typically contain GPS,")
    print("device model, and timestamp data - valuable forensic evidence.")

# Create a sample image
create_sample_image("sample.jpg")

# Display image metadata
show_metadata("sample.jpg")

File: sample.jpg
Format: JPEG
Size: (100, 100)
Mode: RGB
No EXIF metadata found (this sample image has none).
Real photos from phones/cameras typically contain GPS,
device model, and timestamp data - valuable forensic evidence.
```

Experiment 13: Write a Python program to automatically generate a structured digital forensic report from investigation findings.

```
import datetime
# Function to generate a forensic investigation report
def generate_forensic_report(case_id, findings, investigator):
    lines = []

    lines.append("DIGITAL FORENSIC INVESTIGATION REPORT")
    lines.append(f"Case ID: {case_id}")
    lines.append(f"Investigator: {investigator}")
    lines.append(f"Report generated: {datetime.datetime.now()}")
    lines.append("-" * 45)

    lines.append("FINDINGS:")

    for i, finding in enumerate(findings, 1):
        lines.append(f"  {i}. {finding}")

    lines.append("-" * 45)

    lines.append("CONCLUSION:")
    lines.append(
        "  Evidence indicates unauthorized access consistent with a"
    )
    lines.append(
        "  brute-force attack. Recommend IP block and password reset."
    )
    lines.append(
        "  This report is suitable for legal review and expert testimony."
    )

    return "\n".join(lines)

# List of forensic findings
findings = [
    "203.0.113.99 attempted 5 failed logins within 20 seconds (log file evidence).",
    "SHA-256 hash of evidence file verified unchanged throughout custody.",
    "Email header traced to a spoofed domain via suspicious relay server."
]

# Generate the forensic report
report = generate_forensic_report(
    "CASE-2026-014",
    findings,
    "Analyst B"
)

# Display the report
print(report)

# Save the report to a text file
with open("forensic_report.txt", "w") as f:
    f.write(report)

print("\nReport saved successfully as forensic_report.txt")

DIGITAL FORENSIC INVESTIGATION REPORT
Case ID: CASE-2026-014
Investigator: Analyst B
Report generated: 2026-07-23 09:18:33.378430
-----
FINDINGS:
  1. 203.0.113.99 attempted 5 failed logins within 20 seconds (log file evidence).
  2. SHA-256 hash of evidence file verified unchanged throughout custody.
  3. Email header traced to a spoofed domain via suspicious relay server.
-----
CONCLUSION:
  Evidence indicates unauthorized access consistent with a
  brute-force attack. Recommend IP block and password reset.
```

This report is suitable for legal review and expert testimony.

Report saved successfully as forensic_report.txt

Experiment 14: Digital Evidence Collection & Cataloguing

```
import os
import hashlib
# Create evidence directory
evidence_dir = "digital_evidence"
os.makedirs(evidence_dir, exist_ok=True)
# Create sample digital evidence
sample_evidence = {
    "email_log.txt": "From: attacker@mail.com\nTo: victim@mail.com\nSubject: Invoice\nPlease pay immediately.",

    "browser_history.txt": "http://malicious-site.com/login\nhttp://bank.com/transfer",

    "chat_message.txt": "Hey, did you send the file?",

    "system_log.txt": "2026-07-24 10:15:32 - USB device connected: E:\\"
}
# Write sample evidence files
for filename, content in sample_evidence.items():
    with open(os.path.join(evidence_dir, filename), "w") as f:
        f.write(content)
# Function to catalog evidence
def catalog_evidence(folder):
    catalog = []
    for fname in sorted(os.listdir(folder)):
        path = os.path.join(folder, fname)
        # Get file metadata
        stat = os.stat(path)
        # Read file contents
        with open(path, "rb") as f:
            data = f.read()
        # Add evidence information to catalog
        catalog.append({
            "filename": fname,
            "size_bytes": stat.st_size,
            "md5": hashlib.md5(data).hexdigest(),
        })
    return catalog
# Generate evidence catalog
evidence_catalog = catalog_evidence(evidence_dir)
# Display evidence catalog
print("=== DIGITAL EVIDENCE CATALOG ===")
for item in evidence_catalog:
    print(item)

=== DIGITAL EVIDENCE CATALOG ===
{'filename': 'browser_history.txt', 'size_bytes': 56, 'md5': '7804eb386588e70e52f8e44985aae9ab'}
{'filename': 'chat_message.txt', 'size_bytes': 27, 'md5': 'e5f0758dbf1fe3c48005ea2560fa5279'}
{'filename': 'email_log.txt', 'size_bytes': 84, 'md5': '94d472f8d614b91afc7ab2f56ddb435d'}
{'filename': 'system_log.txt', 'size_bytes': 47, 'md5': '5731691a0851b79317771c6e09f5c53b'}
```

Experiment 15: Volatile vs Non-Volatile Evidence Capture

```
import os
import psutil
# Evidence directory
evidence_dir = "digital_evidence"
# Create the folder if it does not already exist
os.makedirs(evidence_dir, exist_ok=True)
# Capture volatile evidence
def capture_volatile_evidence():
    processes = [
        p.info
        for p in psutil.process_iter(['pid', 'name'])
    ]
    return {
        "running_process_count": len(processes),
        "sample_processes": processes[:5],
    }
# Capture non-volatile evidence
def capture_nonvolatile_evidence(folder):
    files = sorted(os.listdir(folder))
    return {
        "disk_files": files,
        "file_count": len(files)
    }
# Capture evidence
volatile_snapshot = capture_volatile_evidence()
nonvolatile_snapshot = capture_nonvolatile_evidence(evidence_dir)
# Display results
print("=== DIGITAL FORENSIC EVIDENCE ===")
print(
    "Volatile evidence -> running processes:",
    volatile_snapshot["running_process_count"]
)
print(
    "Sample running processes:",
    volatile_snapshot["sample_processes"]
)
print(
    "Non-volatile evidence -> disk files:",
    nonvolatile_snapshot
)

=== DIGITAL FORENSIC EVIDENCE ===
Volatile evidence -> running processes: 15
Sample running processes: [{'name': 'docker-init', 'pid': 1}, {'name': 'node', 'pid': 7}, {'name': 'oom_monitor.sh', 'pid': 10}, {'name': 'python3', 'pid': 11}, {'name': 'systemd', 'pid': 12}]
Non-volatile evidence -> disk files: {'disk_files': ['browser_history.txt', 'chat_message.txt', 'email_log.txt', 'system_log.txt']}
```


Experiment 16: Physical, Logical and Sparse Acquisition

```
import os
os.makedirs("acquisition_demo/source", exist_ok=True)
source = "acquisition_demo/source"

with open(os.path.join(source, "report.docx"), "w") as f:
    f.write("Confidential quarterly report.")

with open(os.path.join(source, "photo.jpg"), "w") as f:
    f.write("FAKEJPEGDATA")

raw_disk = b"REPORTDOCX_CONTENT" + b"\x00" * 20 + b"DELETED_INVOICE_DATA" + b"\x00" * 10

def physical_acquisition(raw_bytes):
    return bytes(raw_bytes) # every bit: used + unused + deleted

def logical_acquisition(folder):
    return {f: open(os.path.join(folder, f), "rb").read() for f in os.listdir(folder)}

def sparse_acquisition(folder, targets):
    return {f: open(os.path.join(folder, f), "rb").read() for f in targets if f in os.listdir(folder)}

physical_image = physical_acquisition(raw_disk)
logical_image = logical_acquisition(source)
sparse_image = sparse_acquisition(source, ["report.docx"])

print("Physical image size:", len(physical_image), "bytes")
print("Logical image files:", list(logical_image.keys()))
print("Sparse image files:", list(sparse_image.keys()))

Physical image size: 68 bytes
Logical image files: ['report.docx', 'photo.jpg']
Sparse image files: ['report.docx']
```

Experiment 17: Live vs Dead Acquisition

```
import time
import json
import psutil

def live_acquisition():
    return {
        "time": time.time(),
        "processes": len(psutil.pids()),
        "cpu": psutil.cpu_percent()
    }

def dead_acquisition(file):
    with open(file) as f:
        return json.load(f)

with open("snapshot.json", "w") as f:
    json.dump({"disk_files": ["a.txt", "b.txt"], "ram": None}, f)

print("Live:", live_acquisition())
print("Dead:", dead_acquisition("snapshot.json"))

Live: {'time': 1785228805.0676692, 'processes': 17, 'cpu': 19.7}
Dead: {'disk_files': ['a.txt', 'b.txt'], 'ram': None}
```

Experiment 18: Imaging vs Duplication

```
raw_disk_full = (  
    b"FILE1DATA"  
    + b"\x00" * 15  
    + b"DELETED_FILE_DATA"  
    + b"\x00" * 15  
    + b"FREE_SPACE_00000"  
)  
  
def create_forensic_image(raw_bytes):  
    return bytes(raw_bytes)  
  
def create_duplication(raw_bytes, active_regions):  
    return b"".join(  
        raw_bytes[start:end] for start, end in active_regions  
    )  
  
active_regions = [(0, 9)]  
  
forensic_image = create_forensic_image(raw_disk_full)  
duplication_copy = create_duplication(raw_disk_full, active_regions)  
  
print("Forensic image size:", len(forensic_image))  
print("Duplication size:", len(duplication_copy))  
  
Forensic image size: 72  
Duplication size: 9
```

Experiment 19: Bit Stream Copy Verification

```
import os
import hashlib

original_path = "original_evidence.bin"

with open(original_path, "wb") as f:
    f.write(os.urandom(1024))

def bit_stream_copy(src_path, dst_path):
    with open(src_path, "rb") as src:
        with open(dst_path, "wb") as dst:
            dst.write(src.read())

copy_path = "bitstream_copy.bin"
bit_stream_copy(original_path, copy_path)

def sha256_of_file(path):
    with open(path, "rb") as f:
        return hashlib.sha256(f.read()).hexdigest()

original_hash = sha256_of_file(original_path)
copy_hash = sha256_of_file(copy_path)

print("Original hash:", original_hash)
print("Copy hash:      ", copy_hash)
print("Match:", original_hash == copy_hash)

Original hash: 280299f08ca208b9b71024ad19e4e81e55f69431eb57e7c7e32b5bc143651d1c
Copy hash:      280299f08ca208b9b71024ad19e4e81e55f69431eb57e7c7e32b5bc143651d1c
Match: True
```

Experiment 20: Hashing & the Avalanche Effect

```
import hashlib

def compute_hashes(data: bytes):
    return {
        "MD5": hashlib.md5(data).hexdigest(),
        "SHA1": hashlib.sha1(data).hexdigest(),
        "SHA256": hashlib.sha256(data).hexdigest(),
    }

original_data = b"hello"
tampered_data = b"Hello"

hashes_original = compute_hashes(original_data)
hashes_tampered = compute_hashes(tampered_data)

print("hello ->", hashes_original)
print("Hello ->", hashes_tampered)

hello -> {'MD5': '5d41402abc4b2a76b9719d911017c592', 'SHA1': 'aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d', 'SHA256': '2cf243a069fcb06b28be43609cd01e2450a24e06fb6aa77b2300c34c9e1daeb28'}
Hello -> {'MD5': '8b1a9953c4611296a827abf8c47804d7', 'SHA1': 'f7ff9e8b7bb2e09b70935a5d785e0cc5d9d0abf0', 'SHA256': '185f8b3c2273e968ecce05fa2c8307c3131f9147a5016732c1bde014b04117650'}
```

Experiment 21: File Carving (Data Recovery)

```
import os
def make_fake_jpeg(payload: bytes):
    return b"\xff\xd8\xff" + payload + b"\xff\xd9"
jpeg1 = make_fake_jpeg(b"PHOTO_OF_SUSPECT_CAR")
jpeg2 = make_fake_jpeg(b"CCTV_FRAME_CAPTURE")
raw_disk_blob = (
    os.urandom(30) + jpeg1 +
    os.urandom(40) + jpeg2 +
    os.urandom(20)
)
def carve_jpegs(blob: bytes):
    recovered = []
    start_marker = b"\xff\xd8\xff"
    end_marker = b"\xff\xd9"
    pos = 0
    while True:
        start = blob.find(start_marker, pos)
        if start == -1:
            break
        end = blob.find(end_marker, start)
        if end == -1:
            break
        end += len(end_marker)
        recovered.append(blob[start:end])
        pos = end
    return recovered
recovered_files = carve_jpegs(raw_disk_blob)
print("Recovered", len(recovered_files), "file(s) via carving")

Recovered 2 file(s) via carving
```

Experiment 22: File System Feature Simulation (FAT vs NTFS vs EXT)

```
class FATFileEntry:
    def __init__(self, name, size):
        self.name = name
        self.size = size
class NTFSFileEntry:
    def __init__(self, name, size, permissions="rw-r--r--"):
        self.name = name
        self.size = size
        self.permissions = permissions
        self.journal_entry = f"WRITE {name} {size} bytes"
class EXTFileEntry:
    def __init__(self, name, size, permissions="rw-r--r--"):
        self.name = name
        self.size = size
        self.permissions = permissions
        self.journal_entry = f"WRITE {name} {size} bytes"
fat_file = FATFileEntry("data.txt", 1024)
ntfs_file = NTFSFileEntry("data.txt", 1024)
ext_file = EXTFileEntry("data.txt", 1024)
print("FAT has permissions attribute:", hasattr(fat_file, "permissions"))
print("NTFS has permissions attribute:", hasattr(ntfs_file, "permissions"))
print("EXT has permissions attribute:", hasattr(ext_file, "permissions"))
```

```
FAT has permissions attribute: False
NTFS has permissions attribute: True
EXT has permissions attribute: True
```

Experiment 23: Slack Space Calculation

```
CLUSTER_SIZE = 4096 # Typical disk cluster size in bytes
def calculate_slack_space(file_size, cluster_size=CLUSTER_SIZE):
    clusters_needed = -(-file_size // cluster_size)
    allocated_space = clusters_needed * cluster_size
    slack_space = allocated_space - file_size
    return allocated_space, slack_space
file_size = 5000 # bytes
allocated, slack = calculate_slack_space(file_size)
print(
    f"File size: {file_size} bytes -> "
    f"allocated: {allocated} bytes, "
    f"slack space: {slack} bytes"
)

File size: 5000 bytes -> allocated: 8192 bytes, slack space: 3192 bytes
```


Experiment 24: End-to-End Forensic Workflow

```
import hashlib
import json

def acquire(path):
    with open(path, "rb") as f:
        return f.read()

def hash_data(data):
    return hashlib.sha256(data).hexdigest()

def verify_integrity(data, expected_hash):
    return hash_data(data) == expected_hash

def analyze(data, keyword: bytes):
    return keyword in data

def generate_report(case_id, file_hash, keyword_found):
    return {
        "case_id": case_id,
        "sha256": file_hash,
        "suspicious_keyword_found": keyword_found,
        "status": "Evidence Verified" if keyword_found else "No Match",
    }

evidence_path = "workflow_evidence.txt"

with open(evidence_path, "w") as f:
    f.write("Transfer $50000 to account 99881122 immediately, do not report.")

acquired_data = acquire(evidence_path)
evidence_hash = hash_data(acquired_data)
integrity_ok = verify_integrity(acquired_data, evidence_hash)
keyword_found = analyze(acquired_data, b"99881122")

report = generate_report(
    "CASE-2026-014",
    evidence_hash,
    keyword_found
)

print(json.dumps(report, indent=2))

{
  "case_id": "CASE-2026-014",
  "sha256": "5761dda7d7476e1b92c42bcb9454f9c018f7b70adcac81879551e70c6f42f4a1",
  "suspicious_keyword_found": true,
  "status": "Evidence Verified"
}
```

Experiment 25: Phishing Email Analysis

```
import re
from urllib.parse import urlparse
from collections import Counter

# Sample phishing emails
emails = [
    {
        "sender": "support@paypal-secure-verify.com",
        "subject": "URGENT: Verify your account",
        "body": "Click here: http://192.168.10.5/login to verify your account immediately."
    },
    {
        "sender": "security@bank-update-login.com",
        "subject": "Your account is suspended",
        "body": "Your account is suspended. Visit https://bank-update-login.com/verify now."
    },
    {
        "sender": "support@company.com",
        "subject": "Monthly Report",
        "body": "Please find the monthly report attached."
    }
]

# Analyze one email
def analyze_email(email):
    sender = email["sender"]
    subject = email["subject"]
    body = email["body"]

    indicators = []
    urls = []

    # Extract URLs
    urls = re.findall(r"https?://[^\s]+", body)

    # Check urgency words
    if re.search(r"urgent|immediately|suspended|verify", subject + " " + body, re.I):
        indicators.append("Urgency or pressure language")

    # Check URLs
    for url in urls:
        domain = urlparse(url).netloc

        # Check for IP address URL
        if re.match(r"^\d+\.\d+\.\d+\.\d+$", domain):
            indicators.append("URL uses raw IP address")

        # Check suspicious domain keywords
        if any(word in domain.lower() for word in ["secure", "verify", "update", "login"]):
            indicators.append("Suspicious domain naming")

    # Check sender domain
    sender_domain = sender.split("@")[-1]

    if any(word in sender_domain.lower() for word in ["secure", "verify", "update", "login"]):
        indicators.append("Suspicious sender domain")

    # Calculate risk score
    risk_score = len(indicators) * 25

    if risk_score >= 50:
        classification = "PHISHING"
    elif risk_score > 0:
        classification = "SUSPICIOUS"
    else:
        classification = "LEGITIMATE"

    return {
        "sender": sender,
        "subject": subject,
        "urls": urls,
        "indicators": indicators,
```

```

        "risk_score": risk_score,
        "classification": classification
    }

# Analyze all emails
reports = []

for email in emails:
    reports.append(analyze_email(email))

# Display phishing report
print("=== PHISHING EMAIL ANALYSIS REPORT ===")

for i, report in enumerate(reports, 1):
    print(f"\nEmail {i}")
    print("Sender:", report["sender"])
    print("Subject:", report["subject"])
    print("Classification:", report["classification"])
    print("Risk Score:", report["risk_score"], "/ 100")

    print("Extracted URLs:")
    if report["urls"]:
        for url in report["urls"]:
            print(" -", url)
    else:
        print(" - None")

    print("Indicators:")
    if report["indicators"]:
        for indicator in report["indicators"]:
            print(" -", indicator)
    else:
        print(" - None")

# Summary statistics
phishing_count = sum(
    1 for r in reports if r["classification"] == "PHISHING"
)

suspicious_count = sum(
    1 for r in reports if r["classification"] == "SUSPICIOUS"
)

legitimate_count = sum(
    1 for r in reports if r["classification"] == "LEGITIMATE"
)

all_indicators = []

for report in reports:
    all_indicators.extend(report["indicators"])

indicator_count = Counter(all_indicators)

print("\n=== SUMMARY STATISTICS ===")
print("Total Emails:", len(reports))
print("Phishing:", phishing_count)
print("Suspicious:", suspicious_count)
print("Legitimate:", legitimate_count)

print("\nPhishing Tactics Used:")
for tactic, count in indicator_count.items():
    print(f" - {tactic}: {count}")

=== PHISHING EMAIL ANALYSIS REPORT ===

Email 1
Sender: support@paypal-secure-verify.com
Subject: URGENT: Verify your account
Classification: PHISHING
Risk Score: 75 / 100
Extracted URLs:
 - http://192.168.10.5/login

```

Indicators:

- Urgency or pressure language
- URL uses raw IP address
- Suspicious sender domain

Email 2

Sender: security@bank-update-login.com

Subject: Your account is suspended

Classification: PHISHING

Risk Score: 75 / 100

Extracted URLs:

- <https://bank-update-login.com/verify>

Indicators:

- Urgency or pressure language
- Suspicious domain naming
- Suspicious sender domain

Email 3

Sender: support@company.com

Subject: Monthly Report

Classification: LEGITIMATE

Risk Score: 0 / 100

Extracted URLs:

- None

Indicators:

- None

=== SUMMARY STATISTICS ===

Total Emails: 3

Phishing: 2

Suspicious: 0

Legitimate: 1

Phishing Tactics Used:

- Urgency or pressure language: 2
- URL uses raw IP address: 1
- Suspicious sender domain: 2
- Suspicious domain naming: 1

Experiment 26: Malware Hash Analysis

```
import hashlib
import csv

# Sample files
files = {
    "file1.txt": b"Hello World",
    "file2.txt": b"This is a test file",
    "malware.txt": b"Simulated malware sample"
}

# Known malware SHA-256 hashes (example)
malware_db = set()

# Create hash database
with open("malware_hash_database.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Filename", "MD5", "SHA1", "SHA256", "Detection"])

    for name, data in files.items():
        md5 = hashlib.md5(data).hexdigest()
        sha1 = hashlib.sha1(data).hexdigest()
        sha256 = hashlib.sha256(data).hexdigest()

        # Simulate malware database
        if name == "malware.txt":
            malware_db.add(sha256)

        result = "MALWARE" if sha256 in malware_db else "SAFE"

        writer.writerow([name, md5, sha1, sha256, result])

        print(name, "->", result)

print("\nDatabase saved as malware_hash_database.csv")

file1.txt -> SAFE
file2.txt -> SAFE
malware.txt -> MALWARE

Database saved as malware_hash_database.csv
```

Experiment 27: Ransomware IoCs and Signatures

```
import re

# Simulated ransomware IoCs
families = {
    "LockBit": {
        "domains": ["lockbit-example.com"],
        "ips": ["192.0.2.10"],
        "extensions": [".lockbit"],
        "note": "Your files have been encrypted"
    },
    "WannaCry": {
        "domains": ["wannacry-example.com"],
        "ips": ["192.0.2.20"],
        "extensions": [".wncry"],
        "note": "Ooops, your files have been encrypted"
    }
}

# YARA-like detection rules
rules = {
    "LockBit": [b"Your files have been encrypted", b".lockbit"],
    "WannaCry": [b"Ooops, your files have been encrypted", b".wncry"]
}

# Simulated ransomware note
sample = b"Your files have been encrypted. Rename file to document.lockbit"

print("=== RANSOMWARE IOC ANALYSIS ===")

for family, iocs in families.items():
    print("\nFamily:", family)
    print("Domains:", iocs["domains"])
    print("IPs:", iocs["ips"])
    print("Extensions:", iocs["extensions"])

    if any(pattern in sample for pattern in rules[family]):
        print("Detection: POSSIBLE", family)

print("\n=== YARA-LIKE RULES ===")

for family, patterns in rules.items():
    print(f"\nrule {family} {{")
    for pattern in patterns:
        print(f'    strings: "{pattern.decode()}"')
    print("    condition: any of them")
    print("}")

=== RANSOMWARE IOC ANALYSIS ===

Family: LockBit
Domains: ['lockbit-example.com']
IPs: ['192.0.2.10']
Extensions: ['.lockbit']
Detection: POSSIBLE LockBit

Family: WannaCry
Domains: ['wannacry-example.com']
IPs: ['192.0.2.20']
Extensions: ['.wncry']

=== YARA-LIKE RULES ===

rule LockBit {
    strings: "Your files have been encrypted"
    strings: ".lockbit"
    condition: any of them
}

rule WannaCry {
    strings: "Ooops, your files have been encrypted"
    strings: ".wncry"
    condition: any of them
}
```

```
import re

# Simulated attack requests
requests = [
    "login user=admin password=123456",
    "login user=admin password=password",
    "login user=admin password=admin123",
    "search?q=' OR '1'='1'",
    "search?q=hello",
    "file=../../etc/passwd"
]

# Attack signatures
patterns = {
    "Weak Password": r"password=(123456|password|admin123)",
    "SQL Injection": r"('|--|OR\s+['\"]?1[ '\"]?\s*=\s*[ '\"]?1)",
    "Directory Traversal": r"\.\.\/|\.\.\\"
}

print("=== ATTACK PATTERN ANALYSIS ===")

for request in requests:
    detected = []

    for attack, pattern in patterns.items():
        if re.search(pattern, request, re.I):
            detected.append(attack)

    if detected:
        print("\nRequest:", request)
        print("Detected:", ", ".join(detected))
    else:
        print("\nRequest:", request)
        print("Status: Normal")

print("\n=== WAF RULE SUGGESTIONS ===")
print("1. Rate-limit repeated login attempts.")
print("2. Use parameterized SQL queries.")
print("3. Block ../ and ../../ path traversal patterns.")
print("4. Enforce strong password policies.")

=== ATTACK PATTERN ANALYSIS ===

Request: login user=admin password=123456
Detected: Weak Password

Request: login user=admin password=password
Detected: Weak Password

Request: login user=admin password=admin123
Detected: Weak Password

Request: search?q=' OR '1'='1'
Detected: SQL Injection

Request: search?q=hello
Status: Normal

Request: file=../../etc/passwd
Detected: Directory Traversal

=== WAF RULE SUGGESTIONS ===
1. Rate-limit repeated login attempts.
2. Use parameterized SQL queries.
3. Block ../ and ../../ path traversal patterns.
4. Enforce strong password policies.
```